

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Направление: 02.03.01 Математика и компьютерные науки

ОТЧЁТ

О выполнении лабораторных работ
по дисциплине «Методы проектирования баз данных»

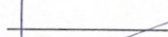
Программирование и настройка СУБД

Студент,
группы 5130201/30002

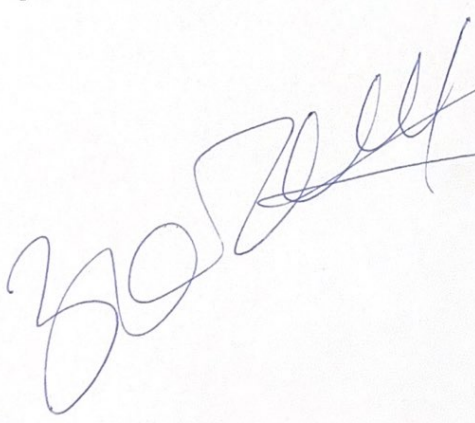


Михайлова А. А.

Преподаватель



Полов С. Г.



«03» 12 2025 г.

Санкт-Петербург, 2025

Реферат

В рамках работы рассматривалось: создание представлений, использование триггеров, управление параллельным доступом к данным через транзакции с различными уровнями изоляции, разграничение прав доступа пользователей на уровне объектов базы данных, а также применение хранимых процедур и функций.

Было создано 1 представление, 6 триггеров, 2 пользователя, 1 процедура и 1 функция. Для выполнения работы использовался диалект MySQL, командная строка и MySQL Workbench 8.0.

Полученная база данных может быть использована в маникюрных салонах для автоматизации учета клиентов, оптимизации графика работы мастеров, анализа востребованности услуг и улучшения качества обслуживания

Содержание

Реферат	2
Введение	4
1 Постановка задачи	5
2 Лабораторные работы	6
2.1 Работа 1: Представления	6
2.2 Работа 2: Триггеры	8
2.3 Работа 3: Транзакции	16
2.4 Работа 4: Права пользователей	18
2.5 Работа 5: Процедуры и функции	20
Заключение	23
Список использованных источников	24
Приложение А. «Исходный текст функции»	25
Приложение А. «Исходный текст процедуры»	26

Введение

База данных (БД) – совокупность данных, хранимых в соответствии со схемой данных, манипулирование которыми выполняют в соответствии с правилами средств моделирования данных.

Система управления базами данных (СУБД) – комплекс программ, позволяющих создать базу данных и манипулировать данными (вставлять, обновлять, удалять и выбирать). Система обеспечивает безопасность, надёжность хранения и целостность данных, а также предоставляет средства для администрирования БД.

Представление – хранимый запрос или результат запроса Select.

Триггер – процедура, которая автоматически запускается при возникновении определенных событий.

Транзакция – это упорядоченная последовательность операторов обработки данных, которая переводит базу данных из одного согласованного состояния в другое.

1 Постановка задачи

В рамках данной работы предусмотрено выполнение пяти лабораторных работ.

- Разработать представление, отображающее результаты запроса с подсчётом количества выполнений каждой услуги каждым мастером. Подготовить 2 запроса, использующих данное представление. Убедиться в невозможности изменения представления.
- Разработать триггеры, срабатывающие при добавлении, удалении и изменении данных.
- Создать 2 пользователей и показать, как они могут взаимодействовать с одними и теми же данными одновременно при помощи транзакций.
- Предоставить созданным пользователям разные права доступа и сравнить результаты после выполнения запросов.
- Реализовать функцию, принимающую в качестве аргументов фамилию, имя и отчество и возвращающую строку формата «Фамилия И.О.» или «Фамилия И.», в зависимости от наличия отчества.
- Реализовать процедуру заполнения базы данных, которая принимает входные параметры из пользовательского интерфейса, проверяет существование записей в связанных таблицах, при необходимости добавляет новые записи, гарантируя целостность данных.

2 Лабораторные работы

2.1 Работа 1: Представления

Задача: Разработать представление для хранения запроса внутри СУБД

Формулировка задачи: Для каждого мастера посчитать количество выполнений каждой услуги

Создано представление `master_service_counts` с посчитанным количеством выполнений каждой услуги для каждого мастера.

```
1 CREATE VIEW master_service_counts AS
2 SELECT
3 M.master_id,
4 T1.service_id,
5 COUNT(T1.okasusluga_id) AS total_services_count
6 FROM
7 master M
8 JOIN
9 schedule Sch ON M.master_id = Sch.master_id
10 JOIN
11 okasusluga T1 ON Sch.schedule_id = T1.schedule_id
12 GROUP BY
13 M.master_id,
14 T1.service_id
15 ORDER BY
16 M.master_id,
17 total_services_count DESC;
```

Представление создает таблицу с колонками `master_id`, `service_id`, `total_services_count`.

master_id	service_id	total_services_count
1	88	44
1	52	44
1	38	43
1	56	43
1	94	43
1	62	43
1	2	43
1	53	41

Рис. 1: Содержание представления

Напишем простой запрос, который использует представление и узнает какие мастера исполняли услугу `n` больше, чем `m` раз.

```
1 SELECT
2 *
```

```

3 FROM
4 master_service_counts
5 WHERE
6 total_services_count > 40
7 AND service_id = 1;

```

master_id	service_id	total_services_count
19	1	43
93	1	41

Рис. 2: Результаты выполнения запроса

Теперь напишем более сложный запрос, который найдет мастеров-чемпионов по каждой услуге и количество её выполнений.

```

1 SELECT
2 MV.service_id,
3 MV.master_id AS Champion_Master_ID,
4 MV.total_services_count AS Max_Count
5 FROM
6 master_service_counts MV
7 JOIN
8 (
9 SELECT
10 service_id,
11 MAX(total_services_count) AS Max_Service_Count
12 FROM
13 master_service_counts
14 GROUP BY
15 service_id
16 ) AS MaxCounts ON MV.service_id = MaxCounts.service_id
17 AND MV.total_services_count = MaxCounts.
18     Max_Service_Count
19 ORDER BY
20 MV.service_id;

```

Проверим что представление доступно только для чтения.

```

1         INSERT INTO master_service_counts (
2     master_id, service_id, total_services_count)
3         VALUES (1, 2, 1);

```

service_id	Champion_Master_...	Max_Count
1	19	43
2	1	43
3	46	48
4	73	40
5	33	41
6	18	42
7	27	42
8	93	48
9	94	44
10	27	44
11	82	40
12	40	41

Рис. 3: Результаты выполнения запроса

✖ 70 00:08:53 INSERT INTO master_service_counts (master_id... Error Code: 1471. The target table master_service_co... 0.036 sec

Рис. 4: Проверка

2.2 Работа 2: Триггеры

Задача: разработать триггеры, срабатывающие при добавлении, удалении и изменении данных.

Формулировка задачи: нужно собирать статистику о количестве записей на определенное время и поддерживать актуальность данных в таблицах.

Для начала создаем и заполняем таблицу для работы с триггерами.

```

1 CREATE TABLE triggertable (time_id int primary key,
   time text not null, count int not null);
2 INSERT INTO triggertable (time_id, time, count)
3 SELECT
4     t.time_id,
5     t.time AS slot_time,
6     COUNT(s.schedule_id) AS total_appointments
7 FROM
8     time t
9 LEFT JOIN
10     schedule s ON s.time_id = t.time_id
11 GROUP BY
12     t.time_id, t.time
13 ORDER BY
14     t.time_id;
```



```

[mysql> select * from triggertable;
+-----+-----+-----+
| time_id | time          | count |
+-----+-----+-----+
| 1       | 4:00 - 5:00  | 730   |
| 2       | 5:00 - 6:00  | 730   |
| 3       | 6:00 - 7:00  | 730   |
| 4       | 7:00 - 8:00  | 730   |
| 5       | 8:00 - 9:00  | 730   |
| 6       | 9:00 - 10:00 | 730   |
| 7       | 10:00 - 11:00| 730   |
| 8       | 11:00 - 12:00| 730   |
| 9       | 12:00 - 13:00| 730   |
| 10      | 13:00 - 14:00| 730   |
| 11      | 14:00 - 15:00| 730   |
| 12      | 15:00 - 16:00| 730   |
| 13      | 16:00 - 17:00| 730   |
| 14      | 17:00 - 18:00| 730   |
| 15      | 18:00 - 19:00| 730   |
| 16      | 19:00 - 20:00| 730   |
| 17      | 20:00 - 21:00| 730   |
| 18      | 21:00 - 22:00| 730   |
| 19      | 22:00 - 23:00| 730   |
| 20      | 23:00 - 24:00| 730   |
+-----+-----+-----+
20 rows in set (0,01 sec)

```

Рис. 5: Результат заполнения таблицы

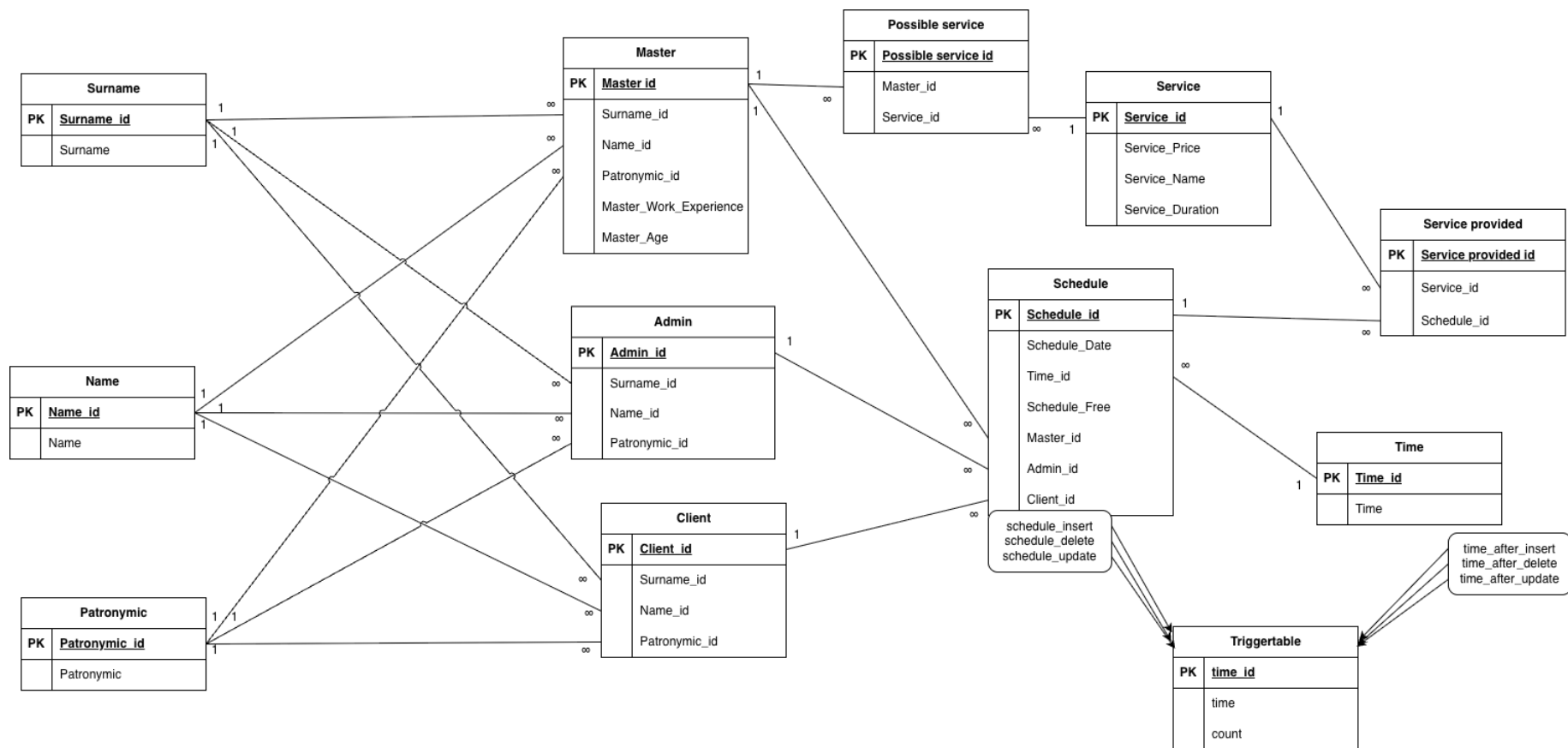


Рис. 6: Диаграмма базы данных на английском языке

Создадим триггер `schedule_insert`, который выполняется после вставки каждой новой записи в таблицу `schedule`. Триггер обновляет таблицу `triggertable`, увеличивая значение `count` на 1 для строк, где `time_id` совпадает со значением `NEW.time_id`.

```
1 DELIMITER $$
2 CREATE TRIGGER schedule_insert
3 AFTER INSERT ON schedule
4 FOR EACH ROW
5 BEGIN
6     UPDATE triggertable
7     SET count = count + 1
8     WHERE time_id = NEW.time_id;
9 END$$
10 DELIMITER ;
11
```

Создадим триггер `schedule_delete`, который выполняется после удаления каждой новой записи из таблицы `schedule`. Триггер обновляет таблицу `triggertable`, уменьшая значение `count` на 1 для строк, где `time_id` совпадает со значением `OLD.time_id`.

```
1 DELIMITER $$
2 CREATE TRIGGER schedule_delete
3 AFTER DELETE ON schedule
4 FOR EACH ROW
5 BEGIN
6     UPDATE triggertable
7     SET count = count - 1
8     WHERE time_id = OLD.time_id;
9 END$$
10 DELIMITER ;
11
```

Код создает триггер `schedule_update`, который срабатывает после изменения записи в таблице `schedule`. Триггер изменяет количество записей `count` в таблице `triggertable`, уменьшая значение на 1 для старого `time_id` (`OLD.time_id`) и увеличивает значение на 1 для нового `time_id` (`NEW.time_id`).

```
1 DELIMITER $$
2 CREATE TRIGGER schedule_update
3 AFTER UPDATE ON schedule
4 FOR EACH ROW
5 BEGIN
6     IF OLD.time_id <> NEW.time_id THEN
7         UPDATE triggertable
```

```

8          SET count = count - 1
9          WHERE time_id = OLD.time_id;
10         UPDATE triggertable
11         SET count = count + 1
12         WHERE time_id = NEW.time_id;
13     END IF;
14 END$$
15 DELIMITER ;
16

```

Создадим триггер time_after_insert, который выполняется после вставки каждой новой записи в таблицу time. Триггер обновляет таблицу triggertable, добавляя новую строку с новым временем и количеством записей 0.

```

1 DELIMITER $$
2 CREATE TRIGGER time_after_insert
3 AFTER INSERT ON time
4 FOR EACH ROW
5 BEGIN
6     INSERT INTO triggertable (time_id, time, count)
7     VALUES (NEW.time_id, NEW.time, 0);
8 END$$
9 DELIMITER ;
10

```

Создадим триггер time_after_delete, который выполняется после удаления записи из таблицы time. Триггер обновляет таблицу triggertable, удаляя старую строку.

```

1 DELIMITER $$
2 CREATE TRIGGER time_after_delete
3 AFTER DELETE ON time
4 FOR EACH ROW
5 BEGIN
6     DELETE FROM triggertable
7     WHERE time_id = OLD.time_id;
8 END$$
9 DELIMITER ;
10

```

Создадим триггер time_after_update, который выполняется после обновления записи в таблице time. Триггер обновляет таблицу triggertable, меняя значение time для определенного time_id.

```

1 DELIMITER $$
2 CREATE TRIGGER time_after_update

```

```

3 AFTER UPDATE ON time
4 FOR EACH ROW
5 BEGIN
6     IF OLD.time <> NEW.time THEN
7         UPDATE triggertable
8         SET time = NEW.time
9         WHERE time_id = NEW.time_id;
10    END IF;
11 END$$
12 DELIMITER ;
13

```

Схема базы данных представлена на рис. 6.

№	Действие и результат																																																															
0	Начальное состояние смежной таблицы <table><tr><th>time_id</th><th>time</th><th>count</th></tr><tr><td>2</td><td>5:00 - 6:00</td><td>730</td></tr><tr><td>3</td><td>6:00 - 7:00</td><td>730</td></tr><tr><td>4</td><td>7:00 - 8:00</td><td>730</td></tr><tr><td>5</td><td>8:00 - 9:00</td><td>730</td></tr><tr><td>6</td><td>9:00 - 10:00</td><td>730</td></tr><tr><td>7</td><td>10:00 - 11:00</td><td>730</td></tr><tr><td>8</td><td>11:00 - 12:00</td><td>730</td></tr><tr><td>9</td><td>12:00 - 13:00</td><td>730</td></tr><tr><td>10</td><td>13:00 - 14:00</td><td>730</td></tr><tr><td>11</td><td>14:00 - 15:00</td><td>730</td></tr><tr><td>12</td><td>15:00 - 16:00</td><td>730</td></tr><tr><td>13</td><td>16:00 - 17:00</td><td>730</td></tr><tr><td>14</td><td>17:00 - 18:00</td><td>730</td></tr><tr><td>15</td><td>18:00 - 19:00</td><td>730</td></tr><tr><td>16</td><td>19:00 - 20:00</td><td>730</td></tr><tr><td>17</td><td>20:00 - 21:00</td><td>730</td></tr><tr><td>18</td><td>21:00 - 22:00</td><td>730</td></tr><tr><td>19</td><td>22:00 - 23:00</td><td>730</td></tr><tr><td>20</td><td>23:00 - 24:00</td><td>730</td></tr><tr><td>100</td><td>4:00 - 5:00</td><td>730</td></tr></table>	time_id	time	count	2	5:00 - 6:00	730	3	6:00 - 7:00	730	4	7:00 - 8:00	730	5	8:00 - 9:00	730	6	9:00 - 10:00	730	7	10:00 - 11:00	730	8	11:00 - 12:00	730	9	12:00 - 13:00	730	10	13:00 - 14:00	730	11	14:00 - 15:00	730	12	15:00 - 16:00	730	13	16:00 - 17:00	730	14	17:00 - 18:00	730	15	18:00 - 19:00	730	16	19:00 - 20:00	730	17	20:00 - 21:00	730	18	21:00 - 22:00	730	19	22:00 - 23:00	730	20	23:00 - 24:00	730	100	4:00 - 5:00	730
time_id	time	count																																																														
2	5:00 - 6:00	730																																																														
3	6:00 - 7:00	730																																																														
4	7:00 - 8:00	730																																																														
5	8:00 - 9:00	730																																																														
6	9:00 - 10:00	730																																																														
7	10:00 - 11:00	730																																																														
8	11:00 - 12:00	730																																																														
9	12:00 - 13:00	730																																																														
10	13:00 - 14:00	730																																																														
11	14:00 - 15:00	730																																																														
12	15:00 - 16:00	730																																																														
13	16:00 - 17:00	730																																																														
14	17:00 - 18:00	730																																																														
15	18:00 - 19:00	730																																																														
16	19:00 - 20:00	730																																																														
17	20:00 - 21:00	730																																																														
18	21:00 - 22:00	730																																																														
19	22:00 - 23:00	730																																																														
20	23:00 - 24:00	730																																																														
100	4:00 - 5:00	730																																																														
1	Добавим в расписание запись с датой 17.10.25, time_id = 7, master_id = 22, admin_id = 1, client_id = 99 <pre>[mysql> select * from triggertable;</pre> <table><tr><th>time_id</th><th>time</th><th>count</th></tr><tr><td>2</td><td>5:00 - 6:00</td><td>730</td></tr><tr><td>3</td><td>6:00 - 7:00</td><td>730</td></tr><tr><td>4</td><td>7:00 - 8:00</td><td>730</td></tr><tr><td>5</td><td>8:00 - 9:00</td><td>730</td></tr><tr><td>6</td><td>9:00 - 10:00</td><td>730</td></tr><tr><td>7</td><td>10:00 - 11:00</td><td>731</td></tr><tr><td>8</td><td>11:00 - 12:00</td><td>730</td></tr><tr><td>9</td><td>12:00 - 13:00</td><td>730</td></tr><tr><td>10</td><td>13:00 - 14:00</td><td>730</td></tr><tr><td>11</td><td>14:00 - 15:00</td><td>730</td></tr><tr><td>12</td><td>15:00 - 16:00</td><td>730</td></tr><tr><td>13</td><td>16:00 - 17:00</td><td>730</td></tr><tr><td>14</td><td>17:00 - 18:00</td><td>730</td></tr><tr><td>15</td><td>18:00 - 19:00</td><td>730</td></tr><tr><td>16</td><td>19:00 - 20:00</td><td>730</td></tr><tr><td>17</td><td>20:00 - 21:00</td><td>730</td></tr><tr><td>18</td><td>21:00 - 22:00</td><td>730</td></tr><tr><td>19</td><td>22:00 - 23:00</td><td>730</td></tr><tr><td>20</td><td>23:00 - 24:00</td><td>730</td></tr><tr><td>100</td><td>4:00 - 5:00</td><td>730</td></tr></table>	time_id	time	count	2	5:00 - 6:00	730	3	6:00 - 7:00	730	4	7:00 - 8:00	730	5	8:00 - 9:00	730	6	9:00 - 10:00	730	7	10:00 - 11:00	731	8	11:00 - 12:00	730	9	12:00 - 13:00	730	10	13:00 - 14:00	730	11	14:00 - 15:00	730	12	15:00 - 16:00	730	13	16:00 - 17:00	730	14	17:00 - 18:00	730	15	18:00 - 19:00	730	16	19:00 - 20:00	730	17	20:00 - 21:00	730	18	21:00 - 22:00	730	19	22:00 - 23:00	730	20	23:00 - 24:00	730	100	4:00 - 5:00	730
time_id	time	count																																																														
2	5:00 - 6:00	730																																																														
3	6:00 - 7:00	730																																																														
4	7:00 - 8:00	730																																																														
5	8:00 - 9:00	730																																																														
6	9:00 - 10:00	730																																																														
7	10:00 - 11:00	731																																																														
8	11:00 - 12:00	730																																																														
9	12:00 - 13:00	730																																																														
10	13:00 - 14:00	730																																																														
11	14:00 - 15:00	730																																																														
12	15:00 - 16:00	730																																																														
13	16:00 - 17:00	730																																																														
14	17:00 - 18:00	730																																																														
15	18:00 - 19:00	730																																																														
16	19:00 - 20:00	730																																																														
17	20:00 - 21:00	730																																																														
18	21:00 - 22:00	730																																																														
19	22:00 - 23:00	730																																																														
20	23:00 - 24:00	730																																																														
100	4:00 - 5:00	730																																																														

№	Действие и результат																																																																		
2	Изменим в добавленной записи time_id на 5: update schedule set time_id = 5 where schedule_id = 14601; <table><tr><th>time_id</th><th>time</th><th>count</th></tr><tr><td>2</td><td>5:00 - 6:00</td><td>730</td></tr><tr><td>3</td><td>6:00 - 7:00</td><td>730</td></tr><tr><td>4</td><td>7:00 - 8:00</td><td>730</td></tr><tr><td>5</td><td>8:00 - 9:00</td><td>731</td></tr><tr><td>6</td><td>9:00 - 10:00</td><td>730</td></tr><tr><td>7</td><td>10:00 - 11:00</td><td>730</td></tr><tr><td>8</td><td>11:00 - 12:00</td><td>730</td></tr><tr><td>9</td><td>12:00 - 13:00</td><td>730</td></tr><tr><td>10</td><td>13:00 - 14:00</td><td>730</td></tr><tr><td>11</td><td>14:00 - 15:00</td><td>730</td></tr><tr><td>12</td><td>15:00 - 16:00</td><td>730</td></tr><tr><td>13</td><td>16:00 - 17:00</td><td>730</td></tr><tr><td>14</td><td>17:00 - 18:00</td><td>730</td></tr><tr><td>15</td><td>18:00 - 19:00</td><td>730</td></tr><tr><td>16</td><td>19:00 - 20:00</td><td>730</td></tr><tr><td>17</td><td>20:00 - 21:00</td><td>730</td></tr><tr><td>18</td><td>21:00 - 22:00</td><td>730</td></tr><tr><td>19</td><td>22:00 - 23:00</td><td>730</td></tr><tr><td>20</td><td>23:00 - 24:00</td><td>730</td></tr><tr><td>100</td><td>4:00 - 5:00</td><td>730</td></tr></table>	time_id	time	count	2	5:00 - 6:00	730	3	6:00 - 7:00	730	4	7:00 - 8:00	730	5	8:00 - 9:00	731	6	9:00 - 10:00	730	7	10:00 - 11:00	730	8	11:00 - 12:00	730	9	12:00 - 13:00	730	10	13:00 - 14:00	730	11	14:00 - 15:00	730	12	15:00 - 16:00	730	13	16:00 - 17:00	730	14	17:00 - 18:00	730	15	18:00 - 19:00	730	16	19:00 - 20:00	730	17	20:00 - 21:00	730	18	21:00 - 22:00	730	19	22:00 - 23:00	730	20	23:00 - 24:00	730	100	4:00 - 5:00	730			
time_id	time	count																																																																	
2	5:00 - 6:00	730																																																																	
3	6:00 - 7:00	730																																																																	
4	7:00 - 8:00	730																																																																	
5	8:00 - 9:00	731																																																																	
6	9:00 - 10:00	730																																																																	
7	10:00 - 11:00	730																																																																	
8	11:00 - 12:00	730																																																																	
9	12:00 - 13:00	730																																																																	
10	13:00 - 14:00	730																																																																	
11	14:00 - 15:00	730																																																																	
12	15:00 - 16:00	730																																																																	
13	16:00 - 17:00	730																																																																	
14	17:00 - 18:00	730																																																																	
15	18:00 - 19:00	730																																																																	
16	19:00 - 20:00	730																																																																	
17	20:00 - 21:00	730																																																																	
18	21:00 - 22:00	730																																																																	
19	22:00 - 23:00	730																																																																	
20	23:00 - 24:00	730																																																																	
100	4:00 - 5:00	730																																																																	
3	Теперь удалим запись: delete from schedule where schedule_id = 14601; <table><tr><th>time_id</th><th>time</th><th>count</th></tr><tr><td>2</td><td>5:00 - 6:00</td><td>730</td></tr><tr><td>3</td><td>6:00 - 7:00</td><td>730</td></tr><tr><td>4</td><td>7:00 - 8:00</td><td>730</td></tr><tr><td>5</td><td>8:00 - 9:00</td><td>730</td></tr><tr><td>6</td><td>9:00 - 10:00</td><td>730</td></tr><tr><td>7</td><td>10:00 - 11:00</td><td>730</td></tr><tr><td>8</td><td>11:00 - 12:00</td><td>730</td></tr><tr><td>9</td><td>12:00 - 13:00</td><td>730</td></tr><tr><td>10</td><td>13:00 - 14:00</td><td>730</td></tr><tr><td>11</td><td>14:00 - 15:00</td><td>730</td></tr><tr><td>12</td><td>15:00 - 16:00</td><td>730</td></tr><tr><td>13</td><td>16:00 - 17:00</td><td>730</td></tr><tr><td>14</td><td>17:00 - 18:00</td><td>730</td></tr><tr><td>15</td><td>18:00 - 19:00</td><td>730</td></tr><tr><td>16</td><td>19:00 - 20:00</td><td>730</td></tr><tr><td>17</td><td>20:00 - 21:00</td><td>730</td></tr><tr><td>18</td><td>21:00 - 22:00</td><td>730</td></tr><tr><td>19</td><td>22:00 - 23:00</td><td>730</td></tr><tr><td>20</td><td>23:00 - 24:00</td><td>730</td></tr><tr><td>100</td><td>4:00 - 5:00</td><td>730</td></tr></table>	time_id	time	count	2	5:00 - 6:00	730	3	6:00 - 7:00	730	4	7:00 - 8:00	730	5	8:00 - 9:00	730	6	9:00 - 10:00	730	7	10:00 - 11:00	730	8	11:00 - 12:00	730	9	12:00 - 13:00	730	10	13:00 - 14:00	730	11	14:00 - 15:00	730	12	15:00 - 16:00	730	13	16:00 - 17:00	730	14	17:00 - 18:00	730	15	18:00 - 19:00	730	16	19:00 - 20:00	730	17	20:00 - 21:00	730	18	21:00 - 22:00	730	19	22:00 - 23:00	730	20	23:00 - 24:00	730	100	4:00 - 5:00	730			
time_id	time	count																																																																	
2	5:00 - 6:00	730																																																																	
3	6:00 - 7:00	730																																																																	
4	7:00 - 8:00	730																																																																	
5	8:00 - 9:00	730																																																																	
6	9:00 - 10:00	730																																																																	
7	10:00 - 11:00	730																																																																	
8	11:00 - 12:00	730																																																																	
9	12:00 - 13:00	730																																																																	
10	13:00 - 14:00	730																																																																	
11	14:00 - 15:00	730																																																																	
12	15:00 - 16:00	730																																																																	
13	16:00 - 17:00	730																																																																	
14	17:00 - 18:00	730																																																																	
15	18:00 - 19:00	730																																																																	
16	19:00 - 20:00	730																																																																	
17	20:00 - 21:00	730																																																																	
18	21:00 - 22:00	730																																																																	
19	22:00 - 23:00	730																																																																	
20	23:00 - 24:00	730																																																																	
100	4:00 - 5:00	730																																																																	
4	Добавим новое время 1:00 - 2:00: insert into time (time) values ("1:00 - 2:00"); <table><tr><th>time_id</th><th>time</th><th>count</th></tr><tr><td>2</td><td>5:00 - 6:00</td><td>730</td></tr><tr><td>3</td><td>6:00 - 7:00</td><td>730</td></tr><tr><td>4</td><td>7:00 - 8:00</td><td>730</td></tr><tr><td>5</td><td>8:00 - 9:00</td><td>730</td></tr><tr><td>6</td><td>9:00 - 10:00</td><td>730</td></tr><tr><td>7</td><td>10:00 - 11:00</td><td>730</td></tr><tr><td>8</td><td>11:00 - 12:00</td><td>730</td></tr><tr><td>9</td><td>12:00 - 13:00</td><td>730</td></tr><tr><td>10</td><td>13:00 - 14:00</td><td>730</td></tr><tr><td>11</td><td>14:00 - 15:00</td><td>730</td></tr><tr><td>12</td><td>15:00 - 16:00</td><td>730</td></tr><tr><td>13</td><td>16:00 - 17:00</td><td>730</td></tr><tr><td>14</td><td>17:00 - 18:00</td><td>730</td></tr><tr><td>15</td><td>18:00 - 19:00</td><td>730</td></tr><tr><td>16</td><td>19:00 - 20:00</td><td>730</td></tr><tr><td>17</td><td>20:00 - 21:00</td><td>730</td></tr><tr><td>18</td><td>21:00 - 22:00</td><td>730</td></tr><tr><td>19</td><td>22:00 - 23:00</td><td>730</td></tr><tr><td>20</td><td>23:00 - 24:00</td><td>730</td></tr><tr><td>21</td><td>1:00 - 2:00</td><td>0</td></tr><tr><td>100</td><td>4:00 - 5:00</td><td>730</td></tr></table>	time_id	time	count	2	5:00 - 6:00	730	3	6:00 - 7:00	730	4	7:00 - 8:00	730	5	8:00 - 9:00	730	6	9:00 - 10:00	730	7	10:00 - 11:00	730	8	11:00 - 12:00	730	9	12:00 - 13:00	730	10	13:00 - 14:00	730	11	14:00 - 15:00	730	12	15:00 - 16:00	730	13	16:00 - 17:00	730	14	17:00 - 18:00	730	15	18:00 - 19:00	730	16	19:00 - 20:00	730	17	20:00 - 21:00	730	18	21:00 - 22:00	730	19	22:00 - 23:00	730	20	23:00 - 24:00	730	21	1:00 - 2:00	0	100	4:00 - 5:00	730
time_id	time	count																																																																	
2	5:00 - 6:00	730																																																																	
3	6:00 - 7:00	730																																																																	
4	7:00 - 8:00	730																																																																	
5	8:00 - 9:00	730																																																																	
6	9:00 - 10:00	730																																																																	
7	10:00 - 11:00	730																																																																	
8	11:00 - 12:00	730																																																																	
9	12:00 - 13:00	730																																																																	
10	13:00 - 14:00	730																																																																	
11	14:00 - 15:00	730																																																																	
12	15:00 - 16:00	730																																																																	
13	16:00 - 17:00	730																																																																	
14	17:00 - 18:00	730																																																																	
15	18:00 - 19:00	730																																																																	
16	19:00 - 20:00	730																																																																	
17	20:00 - 21:00	730																																																																	
18	21:00 - 22:00	730																																																																	
19	22:00 - 23:00	730																																																																	
20	23:00 - 24:00	730																																																																	
21	1:00 - 2:00	0																																																																	
100	4:00 - 5:00	730																																																																	

№	Действие и результат																																																																		
5	Изменим время на 3:00 - 4:00: update time set time = "3:00 - 4:00"where time_id = 21; <table><tr><th>time_id</th><th>time</th><th>count</th></tr><tr><td>2</td><td>5:00 - 6:00</td><td>730</td></tr><tr><td>3</td><td>6:00 - 7:00</td><td>730</td></tr><tr><td>4</td><td>7:00 - 8:00</td><td>730</td></tr><tr><td>5</td><td>8:00 - 9:00</td><td>730</td></tr><tr><td>6</td><td>9:00 - 10:00</td><td>730</td></tr><tr><td>7</td><td>10:00 - 11:00</td><td>730</td></tr><tr><td>8</td><td>11:00 - 12:00</td><td>730</td></tr><tr><td>9</td><td>12:00 - 13:00</td><td>730</td></tr><tr><td>10</td><td>13:00 - 14:00</td><td>730</td></tr><tr><td>11</td><td>14:00 - 15:00</td><td>730</td></tr><tr><td>12</td><td>15:00 - 16:00</td><td>730</td></tr><tr><td>13</td><td>16:00 - 17:00</td><td>730</td></tr><tr><td>14</td><td>17:00 - 18:00</td><td>730</td></tr><tr><td>15</td><td>18:00 - 19:00</td><td>730</td></tr><tr><td>16</td><td>19:00 - 20:00</td><td>730</td></tr><tr><td>17</td><td>20:00 - 21:00</td><td>730</td></tr><tr><td>18</td><td>21:00 - 22:00</td><td>730</td></tr><tr><td>19</td><td>22:00 - 23:00</td><td>730</td></tr><tr><td>20</td><td>23:00 - 24:00</td><td>730</td></tr><tr><td>21</td><td>3:00 - 4:00</td><td>0</td></tr><tr><td>100</td><td>4:00 - 5:00</td><td>730</td></tr></table>	time_id	time	count	2	5:00 - 6:00	730	3	6:00 - 7:00	730	4	7:00 - 8:00	730	5	8:00 - 9:00	730	6	9:00 - 10:00	730	7	10:00 - 11:00	730	8	11:00 - 12:00	730	9	12:00 - 13:00	730	10	13:00 - 14:00	730	11	14:00 - 15:00	730	12	15:00 - 16:00	730	13	16:00 - 17:00	730	14	17:00 - 18:00	730	15	18:00 - 19:00	730	16	19:00 - 20:00	730	17	20:00 - 21:00	730	18	21:00 - 22:00	730	19	22:00 - 23:00	730	20	23:00 - 24:00	730	21	3:00 - 4:00	0	100	4:00 - 5:00	730
time_id	time	count																																																																	
2	5:00 - 6:00	730																																																																	
3	6:00 - 7:00	730																																																																	
4	7:00 - 8:00	730																																																																	
5	8:00 - 9:00	730																																																																	
6	9:00 - 10:00	730																																																																	
7	10:00 - 11:00	730																																																																	
8	11:00 - 12:00	730																																																																	
9	12:00 - 13:00	730																																																																	
10	13:00 - 14:00	730																																																																	
11	14:00 - 15:00	730																																																																	
12	15:00 - 16:00	730																																																																	
13	16:00 - 17:00	730																																																																	
14	17:00 - 18:00	730																																																																	
15	18:00 - 19:00	730																																																																	
16	19:00 - 20:00	730																																																																	
17	20:00 - 21:00	730																																																																	
18	21:00 - 22:00	730																																																																	
19	22:00 - 23:00	730																																																																	
20	23:00 - 24:00	730																																																																	
21	3:00 - 4:00	0																																																																	
100	4:00 - 5:00	730																																																																	
6	Удалим время 3:00 - 4:00: delete from time where time_id = 21; <table><tr><th>time_id</th><th>time</th><th>count</th></tr><tr><td>2</td><td>5:00 - 6:00</td><td>730</td></tr><tr><td>3</td><td>6:00 - 7:00</td><td>730</td></tr><tr><td>4</td><td>7:00 - 8:00</td><td>730</td></tr><tr><td>5</td><td>8:00 - 9:00</td><td>730</td></tr><tr><td>6</td><td>9:00 - 10:00</td><td>730</td></tr><tr><td>7</td><td>10:00 - 11:00</td><td>730</td></tr><tr><td>8</td><td>11:00 - 12:00</td><td>730</td></tr><tr><td>9</td><td>12:00 - 13:00</td><td>730</td></tr><tr><td>10</td><td>13:00 - 14:00</td><td>730</td></tr><tr><td>11</td><td>14:00 - 15:00</td><td>730</td></tr><tr><td>12</td><td>15:00 - 16:00</td><td>730</td></tr><tr><td>13</td><td>16:00 - 17:00</td><td>730</td></tr><tr><td>14</td><td>17:00 - 18:00</td><td>730</td></tr><tr><td>15</td><td>18:00 - 19:00</td><td>730</td></tr><tr><td>16</td><td>19:00 - 20:00</td><td>730</td></tr><tr><td>17</td><td>20:00 - 21:00</td><td>730</td></tr><tr><td>18</td><td>21:00 - 22:00</td><td>730</td></tr><tr><td>19</td><td>22:00 - 23:00</td><td>730</td></tr><tr><td>20</td><td>23:00 - 24:00</td><td>730</td></tr><tr><td>100</td><td>4:00 - 5:00</td><td>730</td></tr></table>	time_id	time	count	2	5:00 - 6:00	730	3	6:00 - 7:00	730	4	7:00 - 8:00	730	5	8:00 - 9:00	730	6	9:00 - 10:00	730	7	10:00 - 11:00	730	8	11:00 - 12:00	730	9	12:00 - 13:00	730	10	13:00 - 14:00	730	11	14:00 - 15:00	730	12	15:00 - 16:00	730	13	16:00 - 17:00	730	14	17:00 - 18:00	730	15	18:00 - 19:00	730	16	19:00 - 20:00	730	17	20:00 - 21:00	730	18	21:00 - 22:00	730	19	22:00 - 23:00	730	20	23:00 - 24:00	730	100	4:00 - 5:00	730			
time_id	time	count																																																																	
2	5:00 - 6:00	730																																																																	
3	6:00 - 7:00	730																																																																	
4	7:00 - 8:00	730																																																																	
5	8:00 - 9:00	730																																																																	
6	9:00 - 10:00	730																																																																	
7	10:00 - 11:00	730																																																																	
8	11:00 - 12:00	730																																																																	
9	12:00 - 13:00	730																																																																	
10	13:00 - 14:00	730																																																																	
11	14:00 - 15:00	730																																																																	
12	15:00 - 16:00	730																																																																	
13	16:00 - 17:00	730																																																																	
14	17:00 - 18:00	730																																																																	
15	18:00 - 19:00	730																																																																	
16	19:00 - 20:00	730																																																																	
17	20:00 - 21:00	730																																																																	
18	21:00 - 22:00	730																																																																	
19	22:00 - 23:00	730																																																																	
20	23:00 - 24:00	730																																																																	
100	4:00 - 5:00	730																																																																	

2.3 Работа 3: Транзакции

Задача: изучить работу параллельных транзакций.

Формулировка задачи: нужно создать двух пользователей и показать, как они могут взаимодействовать с одними и теми же данными одновременно.

Были созданы 2 пользователя.

```
1 create user 'user1'@'localhost' identified by 'qwerty';
2 create user 'user2'@'localhost' identified by 'qwerty';
3 GRANT ALL PRIVILEGES on mybase.* to 'user1'@'localhost'
4 ;
5 GRANT ALL PRIVILEGES on mybase.* to 'user2'@'localhost'
  ;
```

Выбрана аномалия «неповторяемое чтение», которая возникает когда один пользователь за одну транзакцию дважды читает одни и те же данные, но получает разные результаты из-за изменений другого пользователя. В качестве уровня изоляции использовался READ COMMITTED.

Результат работы – таблица с пошаговым описанием всех запросов и полученных результатов при параллельной работе двух пользователей с базой данных.

№	user1	user2	Результат
1	SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;	SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;	Query OK
2	START TRANSACTION;		Query OK
3	SELECT service_name, service_price FROM service WHERE service_id = 1;		Услуга 1, 2871
4		START TRANSACTION	Query OK
5		UPDATE service SET service_price = 2525 WHERE service_id = 1;	1 row affected
6		COMMIT;	Query OK
7	SELECT service_name, service_price FROM service WHERE service_id = 1;		Услуга 1, 2525
8	COMMIT;		Query OK
9	START TRANSACTION;		Query OK

№	user1	user2	Результат
10	SELECT name FROM name WHERE name_id = 1;		Имя 1
11		START TRANSACTION	Query OK
12		START TRANSACTION; UPDATE name SET name = 'Смирнов' WHERE name_id = 1;	1 row affected
13		COMMIT;	Query OK
14	SELECT name FROM name WHERE name_id = 1;		Смирнов
15	COMMIT;		Query OK

Аномалия «неповторяемое чтение» возникает при использовании уровня изоляции READ COMMITTED. Пользователь user1, выполнив запрос выборки к таблице service на шаге 3, получил значение цены услуги «Услуга 1» 2871, но выполнив тот же самый запрос повторно в рамках этой же транзакции на шаге 7 пользователь увидел уже новое значение - 2525, так как между запросами на шаге 5 в рамках этой же транзакции пользователь user2 обновил это значение на 2525 и на шаге 6 зафиксировал изменения.

Поскольку уровень изоляции READ COMMITTED разрешает чтение только зафиксированных данных, при повторном обращении к тем же строкам пользователь user1 увидел уже новое значение, несмотря на то, что сам не изменял данные и находился в рамках одной транзакции.

Аналогичный сценарий мы получили при работе с таблицей name (шаги 9-15): name изменилось с «Имя1» на «Смирнов» между двумя чтениями в рамках одной транзакции пользователя user1.

Уровень изоляции READ COMMITTED не защищает от аномалии «неповторяемое чтение».

2.4 Работа 4: Права пользователей

Задача: организация работы двух пользователей

Формулировка задачи: необходимо предоставить пользователям user1 и user2. разные права доступа и сравнить результаты после выполнения запросов.

Предоставим пользователям разные права доступа. user1 сможет только просматривать данные в таблице triggertable, user2 сможет просматривать, изменять, добавлять, удалять в таблицах triggertable, time, schedule.

```
1 GRANT SELECT on mybase.* to 'user1'@localhost;
2 GRANT SELECT, INSERT, UPDATE on mybase.* to 'user2'
3 @localhost;
```

№	user1	user2												
1	Прочитаем данные из таблицы triggertable: select * from triggertable where time_id = 2; Результат: <table> <tr><td>time_id</td><td>time</td><td>count</td></tr> <tr><td>2</td><td>5:00 - 6:00</td><td>730</td></tr> </table>	time_id	time	count	2	5:00 - 6:00	730	Прочитаем данные из таблицы triggertable: select * from triggertable where time_id = 2; Результат: <table> <tr><td>time_id</td><td>time</td><td>count</td></tr> <tr><td>2</td><td>5:00 - 6:00</td><td>730</td></tr> </table>	time_id	time	count	2	5:00 - 6:00	730
time_id	time	count												
2	5:00 - 6:00	730												
time_id	time	count												
2	5:00 - 6:00	730												
2	Вставка данных в таблицу triggertable: INSERT INTO triggertable (time_id, time, count) VALUES (100, '3', 1); Результат: <pre>mysql> INSERT INTO triggertable (time_id, time, count) VALUES (100, '3', 1); ERROR 1142 (42000): INSERT command denied to user 'user1'@'localhost' for table 'triggertable'</pre>	Вставка данных в таблицу triggertable: INSERT INTO triggertable (time_id, time, count) VALUES (100, '3', 1); Результат: <pre>mysql> INSERT INTO triggertable (time_id, time, count) VALUES (100, '3', 1); Query OK, 1 row affected (0,00 sec)</pre>												
3	Удаление данных в таблице triggertable: DELETE from triggertable where time_id = 1; Результат: <pre>mysql> DELETE from triggertable where time_id = 1; ERROR 1142 (42000): DELETE command denied to user 'user1'@'localhost' for table 'triggertable'</pre>	Удаление данных в таблице triggertable: DELETE from triggertable where time_id = 100; Результат: <pre>mysql> DELETE from triggertable where time_id = 100; Query OK, 1 row affected (0,00 sec)</pre>												

№	user1	user2
4	Изменение данных в таблице triggertable: UPDATE triggertable SET time_id = 100 where time = '4:00 - 5:00'; Результат: <pre>mysql> UPDATE triggertable SET time_id = 100 where time = '4:00 - 5:00'; ERROR 1142 (42000): UPDATE command denied to user 'user1'@'localhost' for table 'triggertable'</pre>	Изменение данных в таблице triggertable: UPDATE triggertable SET time_id = 100 where time = '4:00 - 5:00'; Результат: <pre>mysql> UPDATE triggertable SET time_id = 100 where time = '4:00 - 5:00'; Query OK, 1 row affected (0.00 sec) Rows matched: 1 Changed: 1 Warnings: 0</pre>

Таким образом, user1 может видеть все изменения, вносимые user2, но не может самостоятельно влиять на данные. При этом user2 обладает всеми необходимыми правами для управления данными, которые затрагивают работу триггеров.

2.5 Работа 5: Процедуры и функции

Задача: изучить работу процедур и функций

Формулировка задачи: необходимо реализовать функцию, принимающую в качестве аргументов фамилию, имя и отчество и возвращает строку формата «Фамилия И.О.» или «Фамилия И.», в зависимости от наличия отчества. Реализовать процедуру заполнения базы данных, которая принимает входные параметры из пользовательского интерфейса, проверяет существование записей в связанных таблицах, при необходимости добавляет новые записи, гарантируя целостность данных.

Реализуем функцию. Код реализации функции представлен в Листинге:

2.5.

Примеры вызова функции:

```
1 SELECT format_fio('Иванов', 'Иван', 'Иванович');  
2 SELECT format_fio('Петров', 'Петр', NULL);
```

format_fio('Иванов', 'Иван', 'Ивано...
Иванов И. И.

Рис. 7: Результат вызова функции локально

format_fio('Петров', 'Петр', N...
Петров П.

Рис. 8: Результат вызова функции локально

```
1 SELECT  
2     m.master_id ,  
3     s.surname ,  
4     n.name ,  
5     p.patronymic ,  
6     format_fio(s.surname , n.name , p.patronymic) AS  
7     formatted_fio  
8 FROM  
9     master m  
10    JOIN  
11        surname s ON m.surname_id = s.surname_id  
12    JOIN  
13        name n ON m.name_id = n.name_id  
14 LEFT JOIN
```

```
patronymic p ON m.patronymic_id = p.
patronymic_id;
```

master_id	surname	name	patronymic	formatted_fio
1	Фамилия 283	Имя 210	NULL	Фамилия 283 И.
2	Фамилия 93	Имя 75	Отчество 148	Фамилия 93 И. О.
3	Фамилия 65	Имя 61	Отчество 190	Фамилия 65 И. О.
4	Фамилия 89	Имя 84	Отчество 166	Фамилия 89 И. О.
5	Фамилия 460	Имя 104	Отчество 97	Фамилия 460 И. О.
6	Фамилия 20	Имя 212	Отчество 43	Фамилия 20 И. О.
7	Фамилия 363	Имя 262	Отчество 19	Фамилия 363 И. О.
8	Фамилия 375	Имя 143	Отчество 53	Фамилия 375 И. О.
9	Фамилия 313	Имя 18	Отчество 196	Фамилия 313 И. О.
10	Фамилия 185	Имя 59	Отчество 167	Фамилия 185 И. О.
11	Фамилия 81	Имя 204	Отчество 122	Фамилия 81 И. О.
12	Фамилия 133	Имя 280	Отчество 26	Фамилия 133 И. О.

Рис. 9: Результат вызова функции глобально

Процедура «add_master_if_not_exists» реализует добавление информации о мастере в базу данных. Сначала она проверяет существование введенной фамилии, имени и отчества. Если таких ф.и.о. ранее не было, они добавляются в таблицы под новым id. Далее проверяется существование мастера с таким ф.и.о., опытом работы и возрастом. Если такого мастера не существует, он добавляется в таблицу master, выводится сообщение «Мастер успешно добавлен». Если существует процесс добавления пропускается, выводится сообщение «Мастер уже существует, добавление пропущено».

Реализуем процедуру. Код реализации процедуры представлен в Листинге: 2.5.

```
[mysql> CALL add_master_if_not_exists('Фамилия 283', 'Имя 210', 'Отчество 169', 1, 48);
+-----+
| status                                     |
+-----+
| Мастер уже существует, добавление пропущено |
+-----+
```

Рис. 10: Пример вызова процедуры с уже существующими данными мастера

```
[mysql> CALL add_master_if_not_exists('Пуятта', 'Чтец', 'Михаил', 20, 20);
+-----+
| new_master_id | status                                     |
+-----+
| 103           | Мастер успешно добавлен                   |
+-----+
```

Рис. 11: Пример вызова процедуры с новыми данными мастера

Блок-схема работы процедуры представлена на рис. 12.

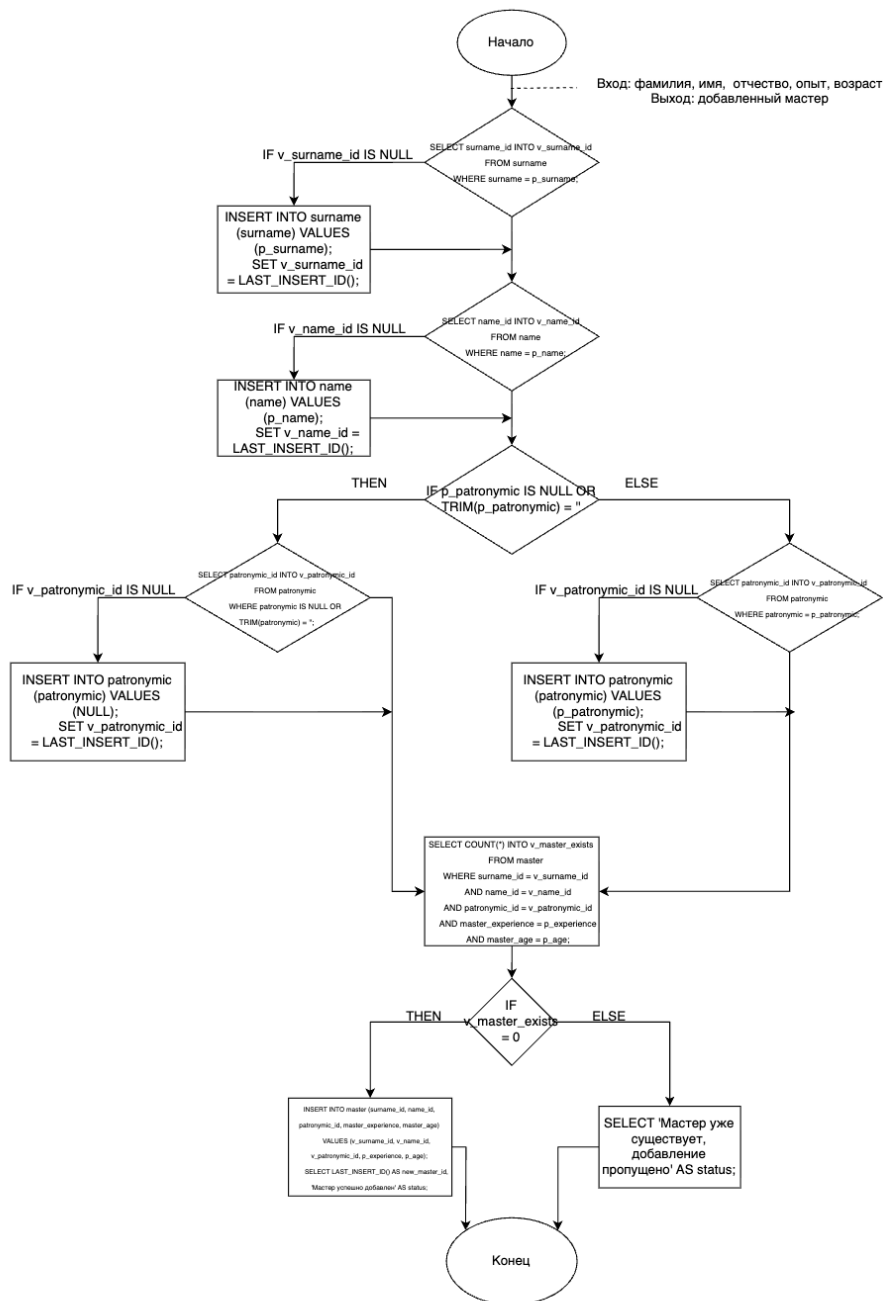


Рис. 12: Блок-схема работы процедуры

Заключение

В рамках работы разработано представление, отображающее результаты запроса, для каждого мастера посчитано количество выполнений каждой услуги. Разработаны триггеры, срабатывающие при добавлении, удалении и изменении данных в таблицах time и schedule. Созданы пользователи с разными правами доступа, работа пользователей продемонстрирована в рамках одной транзакции с одними и теми же данными. Реализованы функция, формирующая строки и процедура, заполняющая базу данных по заданным правилам.

Всего создано 1 представление, 6 триггеров, 2 пользователя, 1 процедура и 1 функция.

Работа выполнялась на устройстве MacBook Pro с чипом Apple M2 Pro и памятью 16ГБ, с использованием диалекта MySQL, приложения draw.io, командной строки и MySQL Workbench 8.0.

Список литературы

- [1] Попов С. Г. Лист самостоятельной проверки хода выполнения курсовой работы по дисциплине «Теоретические основы баз данных» 5 семестр
- [2] Takahashi M., Azuma S., Trend C. L. The manga guide to databases. – No Starch Press, 2009.
- [3] Попов С.Г. Лекции по базам данных. 4 и 5 семестр.

Приложение А. «Исходный текст функции»

```
1 DELIMITER $$
2 CREATE FUNCTION format_fio(
3     p_surname VARCHAR(255),
4     p_name VARCHAR(255),
5     p_patronymic VARCHAR(255)
6 )
7 RETURNS VARCHAR(255)
8 DETERMINISTIC
9 BEGIN
10     DECLARE v_initials VARCHAR(10);
11     DECLARE v_patronymic_initial VARCHAR(10);
12     DECLARE v_result VARCHAR(255);
13     SET v_initials = CONCAT(LEFT(p_name, 1), '.');
14     IF p_patronymic IS NOT NULL AND TRIM(
15         p_patronymic) != '' THEN
16         SET v_patronymic_initial = CONCAT('▢',
17             LEFT(p_patronymic, 1), '.');
18     ELSE
19         SET v_patronymic_initial = '';
20     END IF;
21     SET v_result = CONCAT(p_surname, '▢',
22         v_initials, v_patronymic_initial);
23     RETURN v_result;
24 END$$
25 DELIMITER ;
```

Приложение Б. «Исходный текст процедуры»

```
1 DELIMITER $$
2 CREATE PROCEDURE add_master_if_not_exists(
3     IN p_surname VARCHAR(45),
4     IN p_name VARCHAR(45),
5     IN p_patronymic VARCHAR(45),
6     IN p_experience INT,
7     IN p_age INT
8 )
9 BEGIN
10     DECLARE v_surname_id INT;
11     DECLARE v_name_id INT;
12     DECLARE v_patronymic_id INT;
13     DECLARE v_master_exists INT;
14     SELECT surname_id INTO v_surname_id
15     FROM surname
16     WHERE surname = p_surname;
17     IF v_surname_id IS NULL THEN
18         INSERT INTO surname (surname) VALUES (
19             p_surname);
20         SET v_surname_id = LAST_INSERT_ID();
21     END IF;
22     SELECT name_id INTO v_name_id
23     FROM name
24     WHERE name = p_name;
25     IF v_name_id IS NULL THEN
26         INSERT INTO name (name) VALUES (p_name)
27     ;
28     SET v_name_id = LAST_INSERT_ID();
29     END IF;
30     IF p_patronymic IS NULL OR TRIM(p_patronymic) =
31     ' ' THEN
32         SELECT patronymic_id INTO
33         v_patronymic_id
34         FROM patronymic
35         WHERE patronymic IS NULL OR TRIM(
36         patronymic) = ' ';
37         IF v_patronymic_id IS NULL THEN
38             INSERT INTO patronymic (
39             patronymic) VALUES (NULL);
40             SET v_patronymic_id =
41             LAST_INSERT_ID();
```

```

35         END IF;
36     ELSE
37         SELECT patronymic_id INTO
v_patronymic_id
38         FROM patronymic
39         WHERE patronymic = p_patronymic;
40         IF v_patronymic_id IS NULL THEN
41             INSERT INTO patronymic (
patronymic) VALUES (p_patronymic);
42             SET v_patronymic_id =
LAST_INSERT_ID();
43         END IF;
44     END IF;
45     SELECT COUNT(*) INTO v_master_exists
46     FROM master
47     WHERE surname_id = v_surname_id
48         AND name_id = v_name_id
49         AND patronymic_id = v_patronymic_id
50         AND master_experience = p_experience
51         AND master_age = p_age;
52     IF v_master_exists = 0 THEN
53         INSERT INTO master (surname_id, name_id
, patronymic_id, master_experience, master_age)
54         VALUES (v_surname_id, v_name_id,
v_patronymic_id, p_experience, p_age);
55         SELECT LAST_INSERT_ID() AS
new_master_id, 'Мастер успешно добавлен' AS status;
56     ELSE
57         SELECT 'Мастер уже существует, добавление
пропущено' AS status;
58     END IF;
59 END$$
60 DELIMITER ;

```